Project Report

Placement Portal Application

Student Details

- **Name:** Snehapriya Thiagarajan
- **University:** Indian Institute of Science Education and Research Bhopal
- **Degree:** BS in Electronics and Communication Engineering
- **Registration Number:** 23f3001500

Problem Statement

To build a web based Placement Portal that brings all campus placement activities into a single digital platform . The platform must allow companies to post job openings and review applicants, enabling students to browse opportunities and apply for positions, and provide administrators with tools to supervise and make the overall placement process easy to manage and overlook .

Our Approach

I approached this project by building a web based application that connects students, companies, and the college admin in one place. I used Flask for the backend and a simple Vue.js setup for the frontend. First, I created different dashboards so that Admins can approve Companies and placement drives , Companies can post placement drives , and Students can view the available jobs and apply for them. To make the system work better , I added backend rules that automatically check a student's CGPA before allowing them to apply for a job. Finally, I used background tools like Celery to automatically send update emails and offer letters to students without making the website slow or frozen.

AI/LLM Declaration

AI tools were used in a limited and supportive manner during the development of this project. They were mainly used for debugging, small implementation guidance, and small assistance while designing parts of the frontend user interface. The core system design, database schema, application logic, and overall implementation of the Placement Portal was developed, and verified independently. The use of AI tools is estimated to be approximately 10-15% of the overall development process.

Frameworks and Libraries Used

- Flask – Main backend framework for building REST APIs
- Vue.js – Frontend framework for creating the Single Page Application
- Vue Router – Takes care of the frontend routing and navigation without page reloads
- Flask SQLAlchemy – ORM used for interacting with the SQLite database.
- SQLite - database for storing application data.
- Jinja2 -Template engine used to make the main pages
- Bootstrap 5 -CSS framework used for good design and layout

- Chart.js – Library used for creating charts
- Flask JWT Extended – Provides secure JWT-based authentication and role-based access control
- Axios – Used for sending asynchronous API requests from frontend to backend
- Celery – Task queue used to run background jobs asynchronously
- Redis – Message broker and caching system used with Celery
- Flask Caching – Library used to implement server side caching

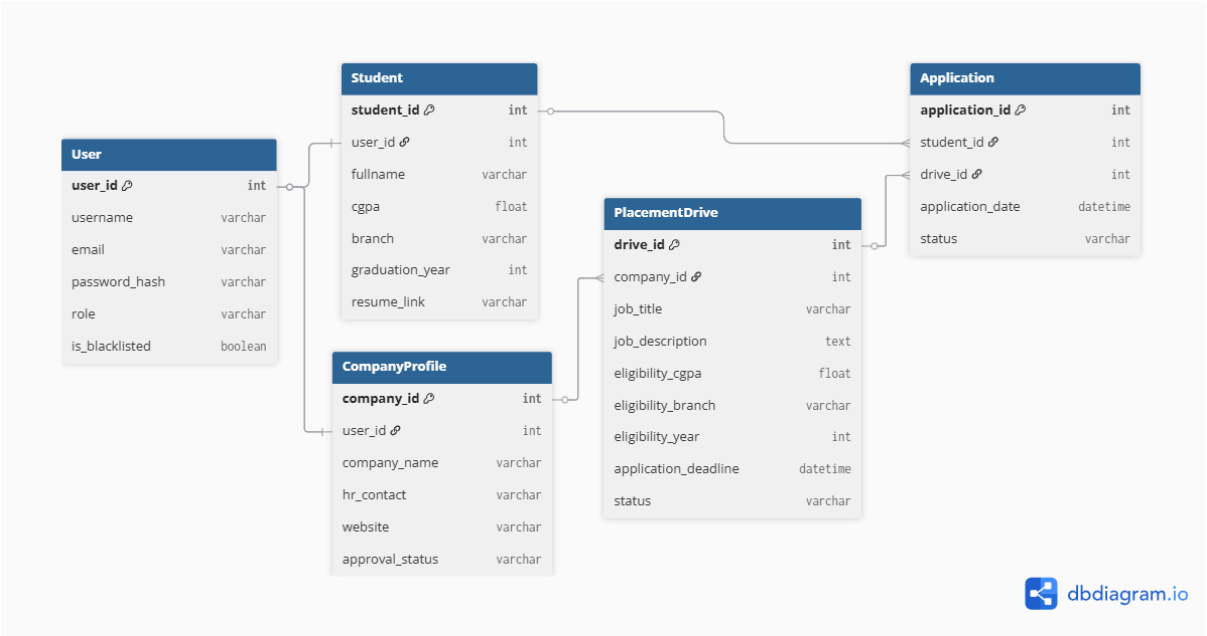
Database Schema / ER Diagram

Tables:

- **User** - stores core authentication details (user_id, username, email, password_hash, role, is_blacklisted)
- **Student** - stores student details (student_id, user_id, fullname, cgpa, branch, graduation_year, resume_link)
- **CompanyProfile** - stores registered company details (company_id, user_id, company_name, hr_contact, website, approval_status)
- **PlacementDrive** - stores job drives created by companies (drive_id, company_id, job_title, job_description, eligibility_cgpa, eligibility_branch, eligibility_year, application_deadline, status)
- **Application** - saves student application history (application_id, student_id, drive_id, application_date, status)

Relationships:

- One-to-One → User → Student
- One-to-One → User → CompanyProfile
- One-to-Many → CompanyProfile → PlacementDrive
- One-to-Many → Student → Application
- One-to-Many → PlacementDrive → Application



API Resource Endpoints

I created RESTful API endpoints for the four main elements of the system: Authentication, Admin actions , Company actions, and Student actions.

- Authentication: Endpoints like "/api/login" and "/api/register" were implemented to check users and make JWT tokens.
- Admin: Endpoints were created to get system statistics and manage approvals .
- Companies & Students: Endpoints were implemented to handle obtaining job lists (GET requests) and submitting data like new job postings or job applications (POST requests).

All protected APIs were implemented using the @jwt_required() decorator to ensure only logged-in users with the correct roles can use them. Data is sent back and forth between the Vue.js frontend and Flask backend in JSON format.

Architecture and Features

Architecture Overview:

- app.py – The main backend file that runs the Flask application, defines the database models using SQLAlchemy, handles background tasks with Celery, and contains all the API routes.
- templates/index.html – The main frontend file where the Single Page Application starts.
- static/app.js- This contains the JS files.
- placement_portal.db – A simple SQLite database file automatically created by SQLAlchemy to store the application data.
- Redis & Celery – Used to manage background tasks and improve performance by handling caching and message passing.

Implemented Features:

- Entry based on user type and login system for different users such as Admin, Students, and Company HR.
- Entry checking using password hashing and JSON Web Tokens to manage user sessions.
- User interface built with Bootstrap 5 that adapts smoothly to both desktop and mobile devices.
- Admin dashboard with charts created using Chart.js.
- Real time search and filtering for users and job listings without requiring page reloads.
- Backend validation rules to ensure conditions such as minimum CGPA are checked before allowing students to apply for jobs.
- Interactive pop up modals that allow users to view detailed student academic profiles and full job descriptions.

Video link

MAD2 PLACEMENT PORTAL 23F3001500